UNIVERSIDAD COOPERATIVA DE COLOMBIA

Ingeniería de software

SIMULADOR DE GESTIÓN DE MEMORIA RAM

Implementación y comparación de los algoritmos First Fit y Best Fit

Presentado por:

Giovanny Poveda Fonseca

Presentado a:

Sergio Luis Lubo Argumedo

Asignatura: Sistemas Operativos

Santa Marta, Magdalena

2026

CONTENIDO

Resumen

1. Introducción

2. Planteamiento del problema y justificación

3. Objetivos

4. Marco conceptual

5. Arquitectura y tecnologías

6. Diseño e implementación

7. Interfaz de usuario

8. Metodología experimental

9. Desarrollo de los escenarios

10. Resultados consolidados

11. Análisis de resultados

12. Limitaciones, riesgos y responsabilidades profesionales

13. Mejoras futuras

14. Conclusiones

15. Referencias

RESUMEN

Este informe documenta el diseño, implementación y evaluación de un simulador didáctico de gestión de memoria mediante asignación contigua. La memoria de prueba se establece en 1024 KB. El sistema permite asignar y liberar procesos, representar bloques libres y ocupados y calcular indicadores de utilización y fragmentación externa. Se implementan dos estrategias: First Fit y Best Fit.

La aplicación utiliza React en el frontend y Flask en el backend, comunicados mediante una API REST. Se evaluaron cuatro escenarios: tres con distribuciones iniciales controladas y uno con una secuencia larga de asignaciones y liberaciones. En el escenario 2, Best Fit asignó tres solicitudes y First Fit asignó dos. En el escenario 3, ambos rechazaron una solicitud de 250 KB pese a existir 450 KB libres en total, porque el mayor bloque libre era de 200 KB. En el escenario 4, sobre 20 secuencias reproducibles de 300 operaciones cada una, Best Fit rechazó en promedio menos solicitudes que First Fit (20,05 frente a 21,90), con una diferencia modesta y no uniforme entre secuencias.

1. INTRODUCCIÓN

La memoria principal es un recurso limitado que el sistema operativo administra para que los procesos dispongan del espacio necesario durante su ejecución. Una tarea de la administración de memoria consiste en decidir dónde ubicar las solicitudes y cómo recuperar el espacio cuando los procesos terminan.

Este proyecto desarrolla un simulador educativo de asignación contigua. Su propósito es representar visualmente cómo diferentes algoritmos seleccionan bloques libres y cómo esas decisiones afectan la utilización y la fragmentación. El programa no administra la RAM física del computador ni reemplaza el administrador de memoria de un sistema operativo real; opera sobre una estructura de datos que representa una memoria hipotética.

2. PLANTEAMIENTO DEL PROBLEMA Y JUSTIFICACIÓN

Cuando los procesos solicitan memoria en distintos momentos, los espacios libres pueden quedar distribuidos en regiones separadas. Puede ocurrir que la suma de los espacios libres sea suficiente para una solicitud, pero que ninguno de ellos tenga por sí solo el tamaño requerido. Esta situación se relaciona con la fragmentación externa.

La comparación de estrategias de asignación permite observar cómo la selección de un bloque influye en la distribución posterior de los espacios libres. El simulador facilita el aprendizaje de conceptos de sistemas operativos mediante una representación visual y experimentos reproducibles.

3. OBJETIVOS

Objetivo general: desarrollar un simulador web de asignación contigua de memoria que permita ejecutar y comparar First Fit y Best Fit mediante escenarios controlados.

Objetivos específicos: representar la memoria como bloques libres y ocupados; implementar asignación y liberación de procesos; unir bloques libres adyacentes; calcular utilización y fragmentación externa; comparar dos algoritmos en cuatro escenarios; y presentar los resultados mediante una interfaz web.

4. MARCO CONCEPTUAL

4.1. Asignación contigua. En el modelo utilizado, cada proceso ocupa un único bloque continuo de direcciones dentro de la memoria simulada. Cada bloque contiene una posición inicial, un tamaño y el identificador del proceso. Si el identificador es nulo, el bloque está libre.

4.2. First Fit. Recorre los bloques en orden y selecciona el primer bloque libre cuyo tamaño sea igual o superior al solicitado. Si el bloque es mayor, se divide entre la región asignada y el remanente libre. Su búsqueda puede detenerse al encontrar el primer espacio adecuado.

4.3. Best Fit. Examina los bloques disponibles y selecciona el bloque adecuado de menor tamaño. Si supera la solicitud, se divide y conserva el remanente como espacio libre. Puede reducir el sobrante inmediato, pero no garantiza mejores resultados para todas las secuencias futuras.

4.4. Fragmentación externa. Se presenta cuando la memoria libre está dividida en varios bloques separados. La suma de esos bloques puede ser suficiente para una solicitud, pero la asignación contigua no puede realizarse si ningún bloque individual alcanza el tamaño requerido.

4.5. Utilización. Es la proporción de la memoria total que se encuentra ocupada por procesos en un instante determinado, expresada como porcentaje.

5. ARQUITECTURA Y TECNOLOGÍAS

El sistema se divide en frontend y backend. El frontend utiliza React para presentar la barra de memoria, el formulario de creación, la tabla de procesos, las estadísticas y la comparación de escenarios. El backend utiliza Python y Flask para recibir solicitudes HTTP, ejecutar la lógica de asignación y devolver el estado en JSON. Flask-CORS permite las solicitudes entre frontend y backend durante el desarrollo.

Las rutas implementadas son GET /api/memoria para consultar el estado; POST /api/procesos para asignar un proceso; DELETE /api/procesos/{proceso_id} para liberar memoria; POST /api/memoria/reiniciar para reiniciar la simulación; y POST /api/comparacion para ejecutar los tres escenarios.

El estado principal se conserva en memoria dentro del proceso backend. No se utiliza una base de datos. Reiniciar el servidor restablece el estado. Los experimentos trabajan con copias de las distribuciones iniciales y no modifican la simulación principal.

6. DISEÑO E IMPLEMENTACIÓN

6.1. Representación. La memoria inicia con un bloque libre de 1024 KB. Cada bloque se representa mediante los campos inicio, tamano y proceso. inicio indica la posición inicial; tamano indica la extensión en KB; proceso identifica al proceso asignado o contiene None si está libre.

6.2. Asignación. El simulador valida que el identificador no esté vacío, que el tamaño sea un entero positivo y que el algoritmo sea válido. Luego invoca la función correspondiente. Si encuentra espacio suficiente, actualiza los bloques; de lo contrario, informa el rechazo.

6.3. División. Cuando un proceso ocupa solo una parte de un bloque libre, el algoritmo crea un bloque ocupado con el tamaño solicitado y otro bloque libre con el espacio sobrante. Así se conserva la suma de tamaños.

6.4. Liberación y unión. Al liberar un proceso, sus bloques se marcan como libres. Después se recorren los bloques y se unen los espacios libres adyacentes. Finalmente se recalculan las posiciones iniciales.

6.5. Métricas. La memoria ocupada es la suma de los tamaños de bloques ocupados. La memoria libre es la memoria total menos la ocupada. También se cuenta el número de bloques libres y se identifica el mayor. Utilización = (memoria ocupada / memoria total) × 100. Fragmentación externa = ((memoria libre total − mayor bloque libre) / memoria libre total) × 100. Si no existe memoria libre, el indicador se establece en cero.

La fórmula de fragmentación es el indicador operativo adoptado en este proyecto para comparar escenarios. No debe interpretarse como una medida universal única: existen distintas formas de caracterizar la fragmentación. Se mantiene una misma definición en pruebas e interfaz para asegurar consistencia.

7. INTERFAZ DE USUARIO

La visualización representa los bloques como segmentos horizontales proporcionales a su tamaño. Los bloques ocupados muestran el identificador del proceso y los KB asignados; los espacios libres se distinguen mediante otro color.

El formulario permite introducir identificador, tamaño en KB y algoritmo. La tabla muestra los procesos asignados y permite liberar cada uno. El panel de estadísticas presenta memoria total, ocupada y libre, utilización, cantidad de bloques libres, mayor bloque libre y fragmentación externa.

La sección de comparación ejecuta los tres escenarios mediante la API y muestra tablas de resultados para ambos algoritmos. El escenario 4 se ejecuta mediante script desde la línea de comandos (Anexo B). La interfaz se adapta a pantallas pequeñas y utiliza un tema oscuro con acentos azules eléctricos.

8. METODOLOGÍA EXPERIMENTAL

Se utilizó una memoria de 1024 KB en los cuatro escenarios. Para cada escenario, ambos algoritmos parten de la misma distribución inicial y reciben las mismas solicitudes en el mismo orden. Cada algoritmo trabaja sobre una copia independiente, por lo que los resultados no se afectan entre sí.

Las métricas registradas son procesos asignados, procesos rechazados, utilización, fragmentación externa y tamaño del mayor bloque libre. Los porcentajes se presentan con dos decimales. Los datos provienen de la ejecución de los scripts incluidos en el repositorio y se

conservan en la carpeta de evidencias. Las instrucciones para reproducirlos se encuentran en el Anexo B.

9. DESARROLLO DE LOS ESCENARIOS

9.1. Escenario 1. Distribución inicial: 300 KB libres, 100 KB ocupados por P2, 280 KB libres, 100 KB ocupados por P4 y 244 KB libres. Se solicita P3 de 250 KB. First Fit selecciona el primer bloque adecuado de 300 KB y deja 50 KB libres en esa región. Best Fit selecciona el bloque de 280 KB y deja 30 KB. Ambos asignan el proceso. La utilización final es 43,95 % en ambos casos. La fragmentación es 51,22 % para First Fit y 47,74 % para Best Fit. El mayor bloque libre es 280 KB para First Fit y 300 KB para Best Fit.

9.2. Escenario 2. Distribución inicial: 200 KB libres, 100 KB ocupados por P1, 300 KB libres, 100 KB ocupados por P2, 100 KB libres y 224 KB ocupados por P3. Se solicitan Q1 de 90 KB, Q2 de 115 KB y Q3 de 190 KB, en ese orden. First Fit asigna dos procesos y rechaza uno; termina con 61,43 % de utilización, 53,16 % de fragmentación y un bloque libre mayor de 185 KB. Best Fit asigna los tres y no rechaza ninguno; termina con 79,98 % de utilización, 46,34 % de fragmentación y un bloque libre mayor de 110 KB. Para esta secuencia Best Fit ubicó más solicitudes, aunque conservó un bloque libre máximo menor.

9.3. Escenario 3. Distribución inicial: 150 KB libres, 100 KB ocupados por P1, 100 KB libres, 100 KB ocupados por P2, 200 KB libres y 374 KB ocupados por P3. Se solicita Q1 de 250 KB. Hay 450 KB libres en total, pero el mayor bloque individual es de 200 KB. Por ello ambos algoritmos rechazan la solicitud. Los dos terminan con cero asignaciones, un rechazo, 56,05 % de utilización, 55,56 % de fragmentación y un bloque libre mayor de 200 KB. El caso demuestra el efecto de la fragmentación externa en asignación contigua.

9.4. Escenario 4. A diferencia de los escenarios 1 a 3, que parten de una memoria fragmentada de forma manual, este escenario inicia con la memoria vacía (1024 KB) y genera la fragmentación mediante el uso. Cada secuencia contiene 300 operaciones pseudoaleatorias: con probabilidad 0,5 se solicita un proceso con tamaño entre 20 y 120 KB y, en caso contrario, se libera un proceso creado previamente. Se evaluaron 20 secuencias con semillas fijas (1 a 20). Para cada semilla, First Fit y Best Fit reciben exactamente la misma secuencia, por lo que el experimento es reproducible. Si una liberación corresponde a un proceso que fue rechazado, se ignora. Al finalizar cada ejecución se verifica que la suma de los bloques sea igual a la

memoria total. Los parámetros (tamaños y probabilidad) fueron elegidos por el equipo para generar una presión moderada sobre la memoria: una configuración inicial con tamaños mayores saturaba la memoria y rechazaba más de la mitad de las solicitudes, lo que distorsionaba la comparación.

Las métricas de utilización, fragmentación externa y mayor bloque libre se calcularon después de cada operación y se promediaron a lo largo de cada secuencia; por tanto, no corresponden solo al estado final. La tabla muestra el promedio sobre las 20 secuencias.

Algoritmo	Rechazos (prom.)	Utilización prom.	Fragmentación prom.	Mayor bloque prom.
First Fit	21,90	51,46 %	36,87 %	367,7 KB
Best Fit	20,05	52,28 %	34,67 %	370,9 KB

Nota: promedios sobre 20 secuencias (semillas 1 a 20). Best Fit rechazó menos solicitudes que First Fit en 11 secuencias, más en 5 y empató en 4.

10. RESULTADOS CONSOLIDADOS

Los resultados se presentan en la tabla comparativa. Las cifras coinciden con las pruebas individuales y la ejecución consolidada de los escenarios 1 a 3. Los resultados del escenario 4, que son promedios sobre 20 secuencias, se presentan en la sección 9.4.

Escenario	Algoritmo	Asignados	Rechazados	Utilización	Fragmentación	Mayor bloque
Escenario 1	First Fit	1	0	43,95 %	51,22 %	280 KB
Escenario 1	Best Fit	1	0	43,95 %	47,74 %	300 KB
Escenario 2	First Fit	2	1	61,43 %	53,16 %	185 KB
Escenario 2	Best Fit	3	0	79,98 %	46,34 %	110 KB
Escenario 3	First Fit	0	1	56,05 %	55,56 %	200 KB
Escenario 3	Best Fit	0	1	56,05 %	55,56 %	200 KB

Nota: porcentajes redondeados a dos decimales. La fragmentación externa corresponde al indicador definido en la sección de implementación.

11. ANÁLISIS DE RESULTADOS

En el escenario 1, ambos algoritmos asignan un proceso del mismo tamaño y alcanzan la misma utilización. La diferencia aparece en la distribución de espacios libres: Best Fit presenta 47,74 % de fragmentación frente a 51,22 % de First Fit.

En el escenario 2, Best Fit asigna más procesos y obtiene mayor utilización. Sin embargo, First Fit conserva un bloque libre máximo de 185 KB, frente a 110 KB de Best Fit. Esto muestra que un indicador favorable no implica que todos los demás también lo sean.

En el escenario 3, ninguno encuentra un bloque individual de 250 KB. La memoria libre total no puede tratarse como un bloque único en el modelo contiguo. El caso ilustra por qué el mayor bloque libre es un dato necesario para comprender el rechazo.

En el escenario 4, Best Fit obtuvo en promedio menos rechazos, menor fragmentación y un mayor bloque libre promedio ligeramente superior (Figura 5). Sin embargo, la diferencia es modesta y no se presenta en todas las secuencias: en 5 de las 20 Best Fit rechazó más solicitudes que First Fit. Esto es coherente con lo observado en el escenario 2 y refuerza que el resultado depende de la secuencia de solicitudes. No se aplicó una prueba estadística, por lo que la ventaja de Best Fit debe entenderse como una tendencia observada en este modelo y con estas semillas, no como una conclusión general.

En conjunto, los experimentos no permiten declarar un ganador universal. Best Fit obtiene mejores resultados en ciertos indicadores y secuencias; First Fit conserva un bloque libre mayor en el escenario 2. La evaluación debe considerar solicitudes atendidas, utilización, fragmentación y tamaño del mayor bloque disponible.

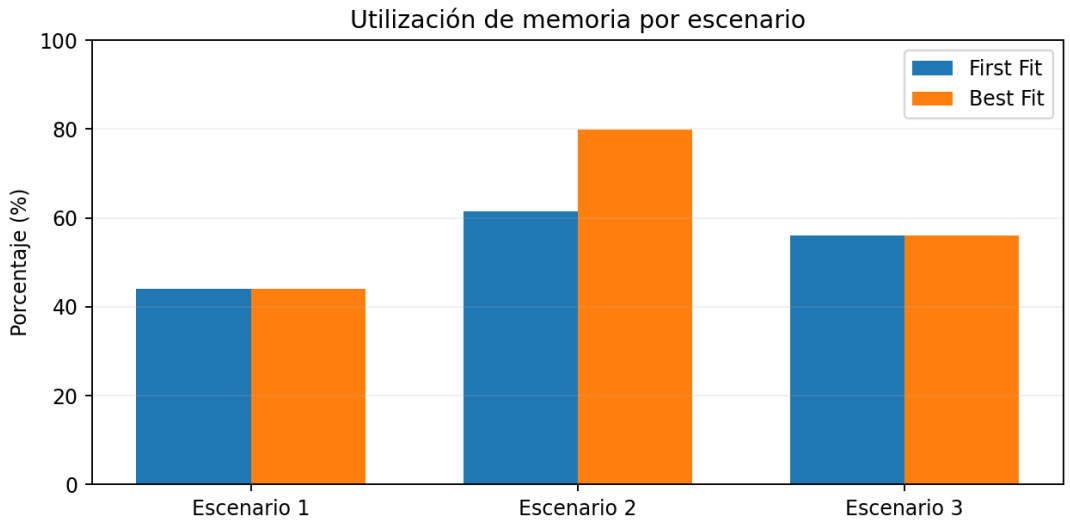


Figura 1. Utilización de memoria por escenario.

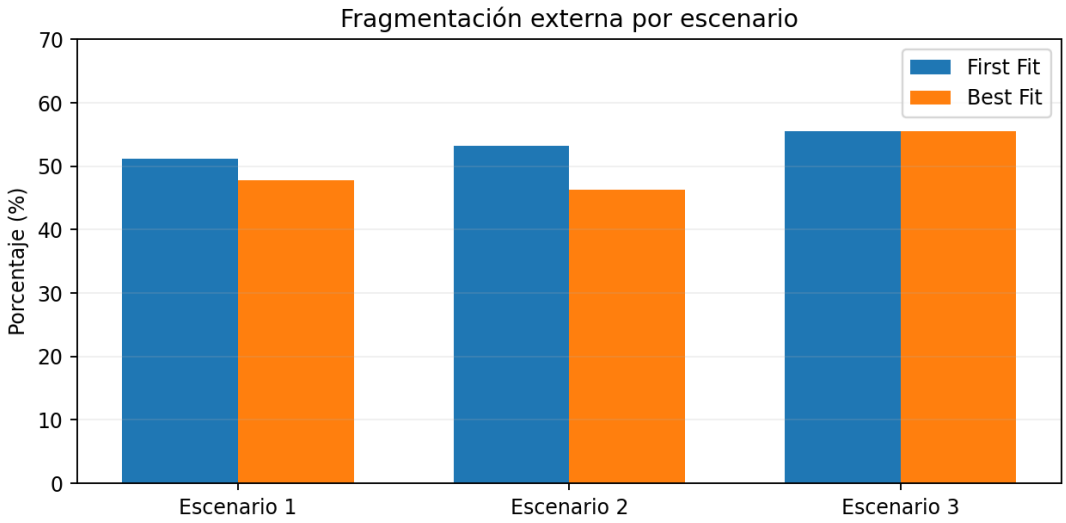


Figura 2. Fragmentación externa por escenario.

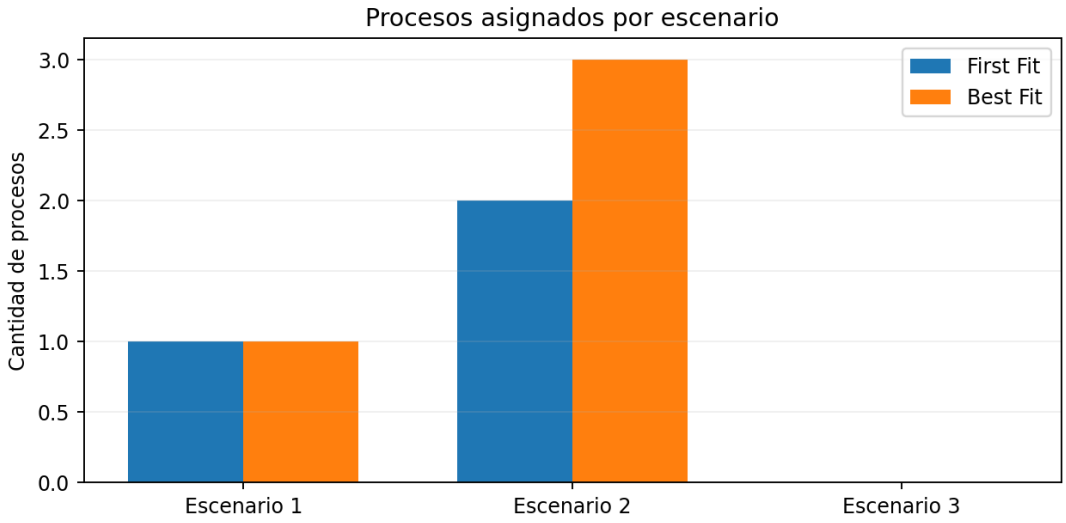


Figura 3. Procesos asignados por escenario.

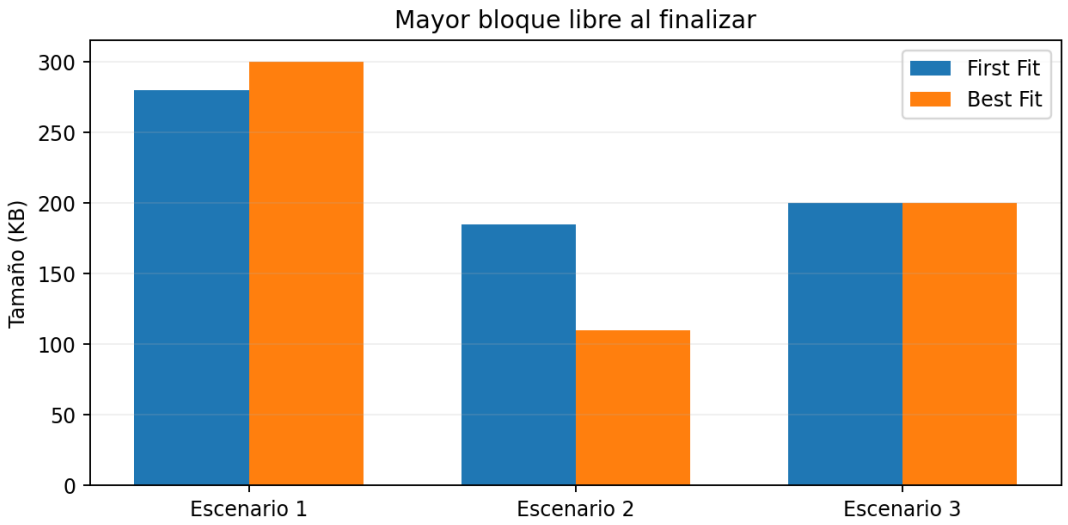


Figura 4. Mayor bloque libre al finalizar cada escenario.

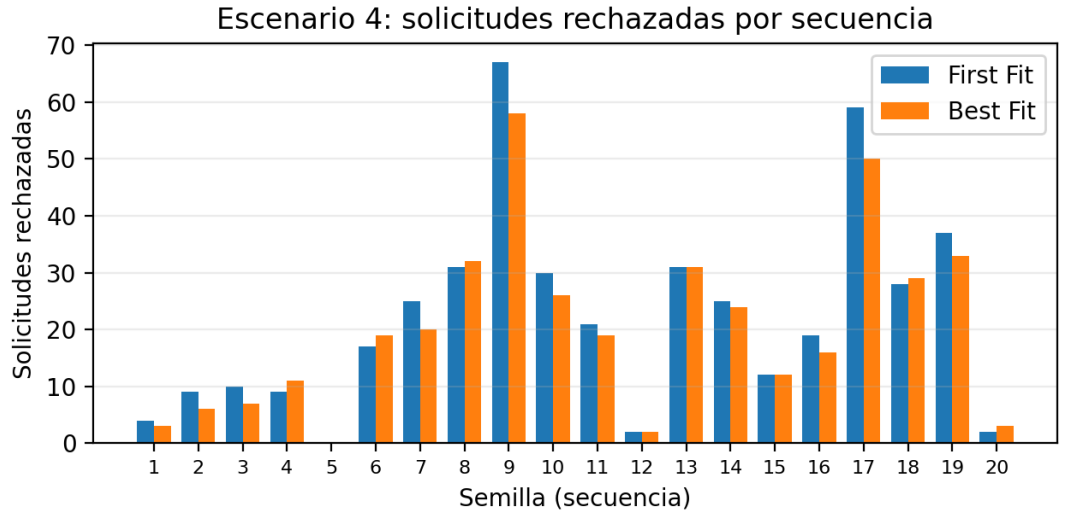


Figura 5. Solicitudes rechazadas por secuencia en el escenario 4.

12. LIMITACIONES, RIESGOS Y RESPONSABILIDADES PROFESIONALES

El simulador representa un modelo simplificado. No implementa memoria virtual, paginación, segmentación, swapping, protección de memoria, concurrencia de procesos ni políticas internas del kernel. No mide tiempos reales de asignación ni la sobrecarga de un sistema operativo.

El estado principal reside en memoria volátil del backend y se pierde al reiniciar el servidor. No hay persistencia histórica, autenticación ni gestión multiusuario. Las solicitudes se expresan en KB enteros.

Los escenarios 1 a 3 parten de distribuciones definidas manualmente y el escenario 4 usa secuencias pseudoaleatorias con parámetros fijados por el equipo; sus resultados dependen de esas elecciones. El número de secuencias es limitado y no se realizó un análisis estadístico.

Riesgos técnicos: entradas incorrectas pueden producir resultados no válidos si no se validan; se mitiga validando identificador, tamaño y algoritmo. Reiniciar el backend borra el estado activo; debe advertirse que la simulación es temporal. Cambios futuros en los algoritmos pueden alterar los resultados; se mitiga manteniendo pruebas reproducibles y revisando las métricas después de cada cambio. Los resultados son de escenarios controlados y no deben generalizarse a cualquier carga de trabajo.

Responsabilidades profesionales

Un simulador que se usa para enseñar o comparar estrategias debe presentar sus resultados con honestidad y sin generalizarlos más allá de lo probado. En este proyecto las conclusiones se limitan a un modelo de asignación contigua con memoria de 1024 KB y a los escenarios ejecutados; no se afirma que un algoritmo sea mejor en sistemas reales. El método se documentó (semillas fijas, mismas secuencias para ambos algoritmos y definición de cada métrica) para que cualquier persona pueda reproducir y verificar las cifras.

Desde el punto de vista del uso responsable de recursos, el simulador opera sobre una estructura de datos y no consume ni modifica la memoria física del equipo. Se utilizaron herramientas de inteligencia artificial como apoyo; el autor es responsable de comprender, verificar y poder explicar el código, los resultados y el análisis presentados.

13. MEJORAS FUTURAS

Como ampliaciones se propone implementar Worst Fit; ampliar el número de secuencias y aplicar un análisis estadístico; permitir escenarios definidos por el usuario; guardar historial; exportar resultados; incorporar gráficos interactivos; y estudiar otros modelos como paginación y segmentación. También podría agregarse persistencia mediante una base de datos.

14. CONCLUSIONES

Se desarrolló un simulador web funcional de asignación contigua que permite crear y liberar procesos, visualizar la distribución de bloques y consultar indicadores. La integración

React-Flask separa la interfaz de la lógica y permite ejecutar operaciones mediante una API REST.

Se implementaron y compararon First Fit y Best Fit en cuatro escenarios reproducibles. Los resultados muestran que el comportamiento depende de la distribución inicial y del orden y tamaño de las solicitudes. Best Fit atendió más procesos en el escenario 2, mientras ambos algoritmos fallaron en el escenario 3 por falta de un bloque contiguo suficientemente grande. En el escenario 4, con secuencias largas de asignaciones y liberaciones, Best Fit mostró en promedio una ventaja modesta que no se presentó en todas las secuencias.

El proyecto permitió observar la diferencia entre memoria libre total y mayor bloque libre, así como la influencia de las decisiones de asignación sobre la fragmentación externa. Las conclusiones se limitan al modelo y a las secuencias probadas.

15. REFERENCIAS

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

Stallings, W. (2018). Operating systems: Internals and design principles (9th ed.). Pearson.

Documentación oficial de Flask: https://flask.palletsprojects.com/

Documentación oficial de React: https://react.dev/

ANEXO A. DISTRIBUCIONES INICIALES DE LOS ESCENARIOS

Escenario 1

300 libre | 100 P2 | 280 libre | 100 P4 | 244 libre. Solicitud: P3 = 250 KB.

Escenario 2

200 libre | 100 P1 | 300 libre | 100 P2 | 100 libre | 224 P3. Solicitudes: Q1 = 90 KB, Q2 = 115 KB, Q3 = 190 KB.

Escenario 3

150 libre | 100 P1 | 100 libre | 100 P2 | 200 libre | 374 P3. Solicitud: Q1 = 250 KB.

Escenario 4

Memoria inicial vacía de 1024 KB. 300 operaciones por secuencia; P(asignar) = 0,5; tamaños entre 20 y 120 KB; 20 secuencias generadas con semillas 1 a 20.

ANEXO B. INSTRUCCIONES DE REPRODUCCIÓN

Requisitos: Python 3.11 o superior; Node.js y npm.

Backend. Desde la carpeta backend: crear un entorno virtual (python -m venv venv), activarlo, ejecutar pip install -r requirements.txt y luego python app.py. La API queda disponible en http://127.0.0.1:5000.

Frontend. Desde la carpeta frontend: npm install y npm start. La interfaz se abre en http://localhost:3000 y requiere el backend en ejecución.

Experimentos sin interfaz. Desde la carpeta backend: python -m tests.comparacion_general (escenarios 1 a 3) y python -m tests.escenario_4_secuencia_larga (escenario 4). Los resultados se guardan en la carpeta evidencias del repositorio. Los scripts deben ejecutarse con la opción -m desde backend para que Python encuentre los paquetes del proyecto.

El README del repositorio contiene el detalle completo, la lista de endpoints y la descripción de las pruebas individuales.